

SVEUČILIŠTE U SPLITU
FAKULTET ELEKTROTEHNIKE, STROJARSTVA I
BRODOGRADNJE

Projektiranje digitalnih sustava

FPGA DETEKCIJA RUBOVA SLIKE U REALNOM
VREMENU

Antonija Grbavac
Ante Šerić
Roko Ante Bogdanović

Split, rujan 2024.

SADRŽAJ

OPIS ZADATKA	2
1. DETEKCIJA RUBOVA NA SLICI.....	3
1.1. Sobel operator	4
2. OPIS IDEJNOG SKLOPA.....	6
2.1. Prijenos piksela slike s računala na FPGA pločicu	6
2.2. Obrada dijelova slike Sobel operatorom.....	7
2.3. Prikaz obrađene slike na VGA zaslonu	8
3. SIMULACIJA	10
4. ZAKLJUČAK.....	15
5. LITERATURA.....	16
DODATAK – pregled programskog koda.....	18

OPIS ZADATKA

Zadatak projekta je FPGA implementacija Sobel operatora za detekciju rubova na slici. Potrebno je osmisliti način prijenosa i obrade veće slike (512 x 512 piksela) na FPGA pločici, opisati korištene metode te osvrnuti se na moguća poboljšanja u implementaciji. Sklop je opisan u Verilog HDL-u, kreiran u Xilinx ISE WebPACK 14.7 razvojnom okruženju, a zbog nemogućnosti implementacije na Spartan-3E XC3S500E pločicu detaljno je opisana i odrađena simulacija logike idejnog rješenja.

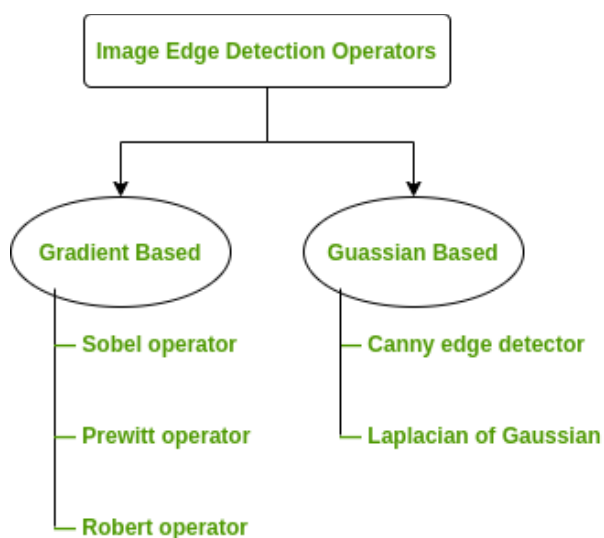
1. DETEKCIJA RUBOVA NA SLICI

Detekcija rubova je metoda segmentiranja slike putem razlikovanja i lociranja područja diskontinuiteta na slici. Široko je korištena tehnika u digitalnoj obradi slike, a primjenu pronalazi u prepoznavanju uzoraka, morfologije slike te izdvajanju značajki.

Rub može sadržavati najvažnije informacije o slici. Sastoji se od skupa piksela, usko povezanih, koji tvore granicu između dva disjunktna područja. Detektiranjem značajnih promjena svjetline piksela područja diskontinuiteta, ova metoda omogućuje jednostavno i precizno promatranje objekata na slici. [2]

Operatore za detekciju rubova na slici moguće je podijeliti u dvije skupine:

- **operatori temeljeni na gradijentu** (engl. *Gradient – based operators*) koji promatraju promjenu intenziteta na rubovima te pronalaze područja gdje je derivacija intenziteta veća od određene granične vrijednosti (poput *Sobel*, *Prewitt*, *Roberts* operatora)
- **Gaussovi operatori** (engl. *Gaussian – based operators*) koji računaju derivacije drugog reda u slici, a detekcijom nultih prijelaza pronalaze rubove (kao *Canny* detektor rubova te *LoG – Laplacian of Gaussian* operator)



Slika 1. Osnovna podjela operatora za detekciju rubova [1]

1.1. Sobel operator

Diskretni je operator diferenciranja. Računa aproksimaciju gradijenta funkcije intenziteta slike za detekciju rubova. Na pikselima slike, Sobelov operator proizvodi ili normalu na vektor ili odgovarajući vektor gradijenta. Koristi dvije 3 x 3 matrice ili maske (M_x i M_y) koje se konvoluiraju s ulaznom slikom za računanje aproksimacije vertikalne i horizontalne derivacije.

$$M_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad M_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Matrica filtra primjenjuje se na dio slike veličine 3 x 3 piksela (blok slike). Svaki element matrice filtra množi se s odgovarajućom vrijednosti piksela iz bloka. Sumiranjem tih umnožaka dobije se vrijednost piksela izlazne slike. Maska se pomiče za jedan piksel udesno te se proces ponavlja za čitavu sliku. Konvolucijom se dobiju dvije slike I_x i I_y :

$$I_x = M_x * A \quad I_y = M_y * A$$

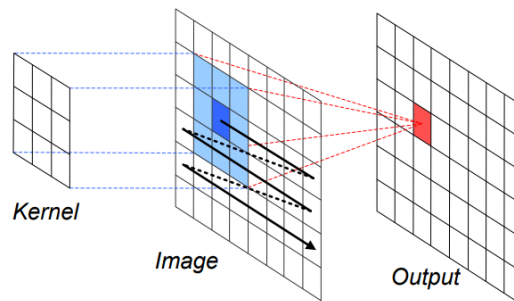
gdje je A matrica piksela izvorne slike. I_x predstavlja promjene intenziteta piksela u horizontalnom smjeru, a I_y promjene intenziteta u vertikalnom smjeru.

Magnituda gradijenta (G_{mag}) te smjer (kut) računaju se prema relacijama

$$|G_{mag}| = \sqrt{I_x^2 + I_y^2}$$

$$\theta(x, y) = \arctg\left(\frac{I_y}{I_x}\right)$$

Magnituda predstavlja jakost ruba, a kut smjer ruba. Kao rezultat dobije se niz rubova na slici, od koji su neki puniji, a neki tanji.



Slika 2. Proces primjene maski operatora na piksele slike [3]

Metoda pruža jednostavan i vremenski učinkovit proračun, a potraga za glatkim rubovima jako je olakšana. Međutim, ovaj operator pokazuje veliku osjetljivost na šum te manju preciznost u detekciji rubova. Dijagonalni rubovi nisu uvijek ispravno sačuvani, a detekcija s jačim i grubim rubovima ne daje odgovarajuće rezultate.



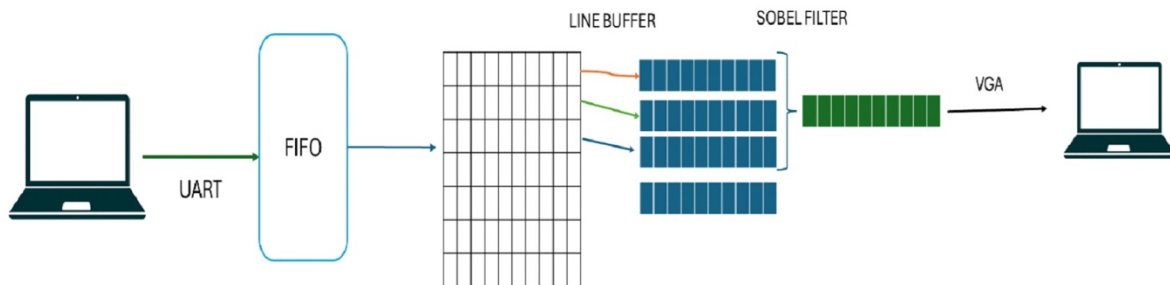
Slika 3. Rezultat obrade slike primjenom Sobel operatora

2. OPIS IDEJNOG SKLOPA

Idejno rješenje koristi složeni algoritam koji se sastoji od nekoliko sklopova skladno ukomponiranih za otkrivanje širokog raspona rubova na slikama. Sastoji se od **3 koraka**:

- prijenos piksela slike s računala na FPGA pločicu
- obrada dijelova slike Sobel operatorom
- prikaz obrađene slike na VGA zaslonu

Sav programski kod, uključujući skripte za transformaciju ulazne te izlazne slike, moguće je pronaći u sekciji *Dodatak* na kraju rada.



Slika 4. Shema implementacije idejnog rješenja

2.1. Prijenos piksela slike s računala na FPGA pločicu

U ovom projektu, prijenos slike postiže se korištenjem UART (engl. *Universal Asynchronous Receiver-Transmitter*) protokola. Podaci o slici šalju se bajt po bajt, sa start i stop bitovima koji označavaju početak i kraj pojedinog bajta. Rad mu je asinkron, što znači da pošiljalatelj i prijemnik mogu koristiti neovisne clock signale. To dodaje fleksibilnost radu sustava, no zahtjeva pažljivu sinkronizaciju. Daje i mogućnost detekcije grešaka u prijenosu, što nije korišteno u ovome radu. No, implementiran je i međusklop za kompenzaciju kako bi se pojava pogrešnih podataka svela na minimum. Uspješno su implementirane i testirane obje inačice (engl. *receiver i transmitter*) UART protokola u slanju velikog niza bajtova, iako je za potrebe projekta potreban samo *receiver*.

Sustav je dizajniran za slike veličine 512 x 512 piksela sivih tonova (olakšava implementaciju) koju je prethodno potrebno pretvoriti u binarne podatke. U tu svrhu kreirana je

Python skripta koja primljenu sliku .jpg formata pretvara u crno – bijele tonove, reže je na tražene dimenzije te piksele pretvara u niz bajtova koji su prikazani kao zasebni retci izlazne tekstualne datoteke.

Asinkroni FIFO međuspremnik olakšava prijenos podataka između dvije sklopovske jedinice koje rade na različitim frekvencijama. Koristi odvojene satove za čitanje i pisanje te adresiranje putem Gray-eva koda za sigurnu sinkronizaciju pokazivača za čitanje i pisanje. FIFO je opremljen signalima za označavanje praznog, punog, gotovo praznog i gotovo punog stanja, sprječavajući „prelijevanje“ podataka (*engl. overflow i underflow*).

2.2. Obrada dijelova slike Sobel operatorom

Kako bi se uskladio prijenos podataka između FIFO međuspremnika i sustava triju linijskih međuspremnika, implementiran je međumodul *uart_fifo2line_buffer*. Prati broj obrađenih bajtova te kontrolira kada se podaci trebaju pročitati iz FIFO međuspremnika pomoću signala *read_req*. Modul radi u različitim stanjima, uključujući inicijalno stanje, uvoz podataka iz FIFO buffera te prijelaz između mirovanja (*engl. idle*) i glavnog stanja (*engl. main*). Koordinira protok podataka na temelju kapaciteta FIFO spremnika i upravljanja prekidima za učinkovitu obradu.

Koristimo četiri line buffera za privremeno pohranjivanje podataka o pikselima i to na način da odmah učitamo fragmente slike u sva četiri. No, odjednom koristimo samo tri sa idejom da nakon što dobijemo kernel obrađene 3x3 Sobel matrice ne gubimo takt već imamo spremnu sljedeću kombinaciju fragmenata slike koji trebaju biti obrađeni Sobel filterom. To osiguravamo takozvanom “kružnom” kombinacijom triju različitih line buffera. Također, valja napomenuti da kao izlaz buffera koristimo 72-bitnu varijablu koja sadrži očitane piksel podatke iz line buffera (3 reda 24-bitnih podataka iz 3 line buffera).

Zbog navedene kružne logike trebamo još implementirati i 4-bitni registar (u kodu *currentWrline_buffer*) koji pokazuje u koji se od 4 line buffer reda trenutno upisuje. Signal *currentRdline_buffer* osigurava da samo trenutni line buffer prima podatke. Za čitanje podataka bitno nam je stanje *rdState* koji kontrolira kada počinje čitanje line buffera.

Sustav se pokreće u stanju mirovanja (IDLE). Kada *totalPixelCounter* (koji prati ukupni broj važećih piksela u svim line bufferima) dosegne 1536 piksela (3*512 piksela), *rdState* prelazi u stanje RD_BUFFER te počinje čitanje iz dostupnih linija. Tijekom ovoga stanja, brojač

rdCounter se povećava kako se podaci čitaju iz line buffera, a kada dođe do 512 piksela, sustav prelazi natrag u IDLE stanje i signalizira prekid (u kodu *o_intr*). Kombinacija line buffera osigurava da izlaz predstavlja piksele iz različitih linija, olakšavajući obradu slike detekcijom rubova.

Izlaz line buffera od 72 bita dolazi na Sobel modul gdje se primjenjuju dvije 3x3 matrice (vertikalni/horizontalni smjer) za određivanje veličine gradijenta intenziteta piksela. Izvodi se operacija konvolucije množenjem podataka sa spomenutim matricama, zbrajanje rezultata te računanje veličine gradijenta. Modul također signalizira valjanost obrađenih podataka putem izlaza *o_convolved_data_valid*.

Modul *sobel2black_white* pretvara rubne piksele sivih tonova u crno-bijeli format koji je nužan za prikaz na VGA ekranu. Uzima 8-bitni ulaz konvoluiranih podataka te daje binarni izlaz. Ako vrijednost intenziteta podatka prelazi definiranu graničnu vrijednost (*engl. threshold*), u ovome slučaju 100, podatak poprima vrijednost 0 (crna boja), odnosno 1 (bijela boja) u suprotnom. Pretvorba se događa samo kada su dostupni valjani podatci (signalizirano *data_valid* ulazom).

2.3. Prikaz obrađene slike na VGA zaslonu

Modul *vga_controller* odgovoran je za generiranje horizontalnih i vertikalnih sinkronizacijskih signala (*o_hs* i *o_vs*) potrebnih za ispravan rad VGA zaslona. Ključni su za sinkronizaciju elektronskog snopa zaslona s podacima o pikselima. Radi s rezolucijom od 512 x 512 piksela, što odgovara veličini izlazne slike, premda je rezolucija VGA ekrana nešto manja (640 x 480 piksela).

Parametri HS_STA, HS_END i HA_STA odgovorni su za horizontalnu sinkronizaciju i određivanje vremena aktivnog piksela. Parametar HA_END definira kraj vodoravne aktivne regije piksela. Slično, parametri VS_STA i VS_END služe vremenskom usklađivanju vertikalne sinkronizacije. Vrijednosti LINE i SCREEN daju informaciju o ukupnoj širini i visini zaslona u linijama. Brojači *h_count* i *v_count* prate trenutnu poziciju piksela. Vraćaju se na vrijednost nula kada dođu do kraja svojih dimenzija, osiguravajući rad i ispravnu sinkronizaciju.

Modul *vga_image* jednostavnija je komponenta koja uzima jedan binarni ulaz (*color_in*) i prevodi ga u RGB boje (*i_VGA_RED*, *i_VGA_GREEN*, *i_VGA_BLUE*) za VGA izlaz. Ovaj modul preslikava binarni unos boje u RGB vrijednost:

- Ako je color_in u jedinici, svi RGB signali postavljeni su na 1, što rezultira bijelom bojom.
- Ako je color_in nula, svi RGB signali postavljeni su na 0, što rezultira crnom bojom.

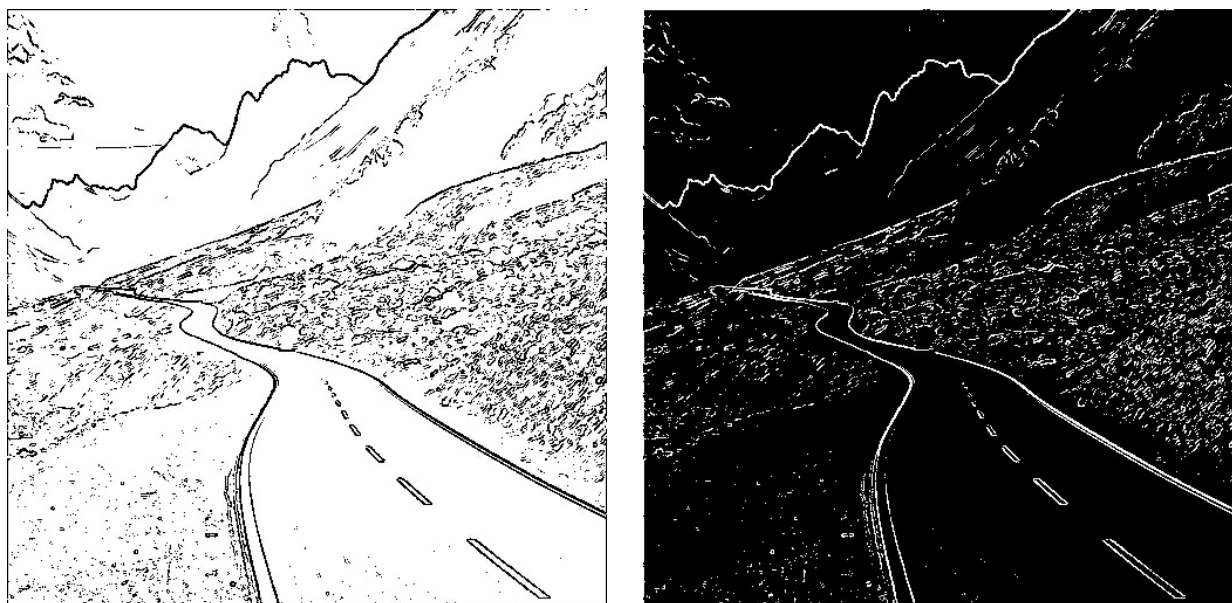
Ovo jednostavno mapiranje modulu omogućuje kreiranje crno-bijelih slika putem VGA izlaza punim intenzitetom ili bez intenziteta za svaki kanal u boji, na temelju ulaznog signala.

3. SIMULACIJA

U simulaciji smo napravili takozvani “*proof of concept*” kojim smo željeli dokazati izvedivost projekta. No, kao važnu napomenu treba istaknuti, da iako smo i u simulaciji izbjegli korištenje for petlji, u implementaciji bi se trebali koristiti shift registri što bi se moglo pokazati kao komplicirano rješenje. U simulaciju uvodimo konstante kao parametre. Pošto obrađujemo 512x512px sliku, uvodimo parametre visine i širine slike kao i parametre IMAGE_WIDTH_E i IMAGE_HIGHT_E koji su u našem slučaju 9 (jer je $2^9 = 512$). Uvodimo i dva bitna lokalna parametra: BYTE_SIZE - kojim bitove grupiramo u bytove, odnosno namještamo ga na fiksnu vrijednost 8; INIT_LINE_NUM_COUNT - kojim inicijaliziramo brojač linija u line bufferu na nulu.

Kao ulaz koristimo binarnu tekstualnu datoteku crno-bijele slike. U ovom kodu koristimo tri line buffera koji drže tri uzastopna retka piksela slike istovremeno, te kao i u prijašnjoj implementacijskoj logici, koristi se Sobel filtriranje sa 3x3 matricom koja prolazi preko vrijednosti piksela line buffera.

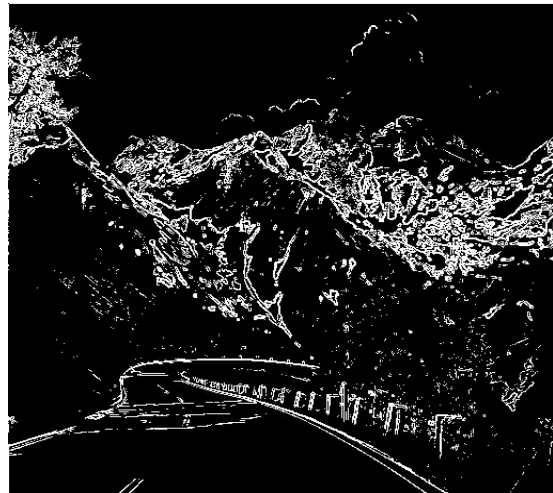
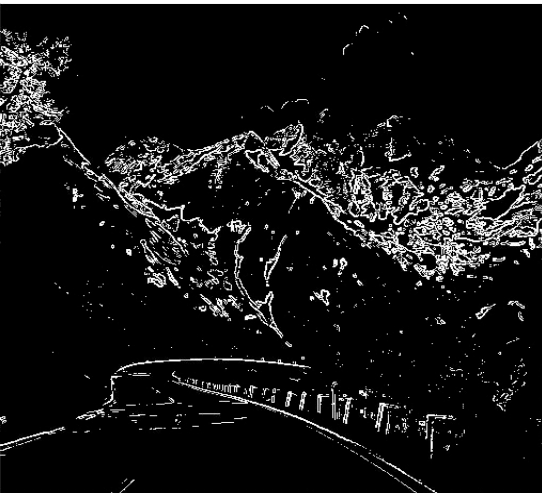
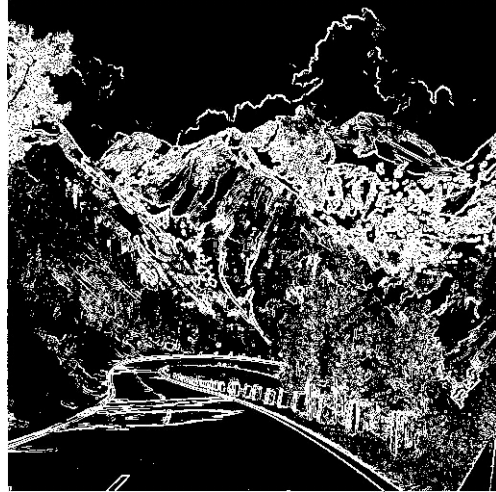
Modul radi na takt (*clk*) i reset (*reset*) ulazima. Slika je pohranjena u 2D polje pod nazivom *original_image*, s dimenzijama definiranim parametrima (IMAGE_WIDTH i IMAGE_HIGHT). Na *Slici 5.* možemo vidjeti rezultate uspješno filtrirane slike Sobel filterom. Jasno se mogu iščitati rubovi ceste, ali i obrisi okoline. Također demonstrirano je kako je lako mijenjati kontrastne boje iz bijele sa crnim rubovima u crnu sa bijelim rubovima.



Slika 5. (s lijeva na desno) slika filtrirana kao bijelo-crna, slika filtrirana kao crno-bijela

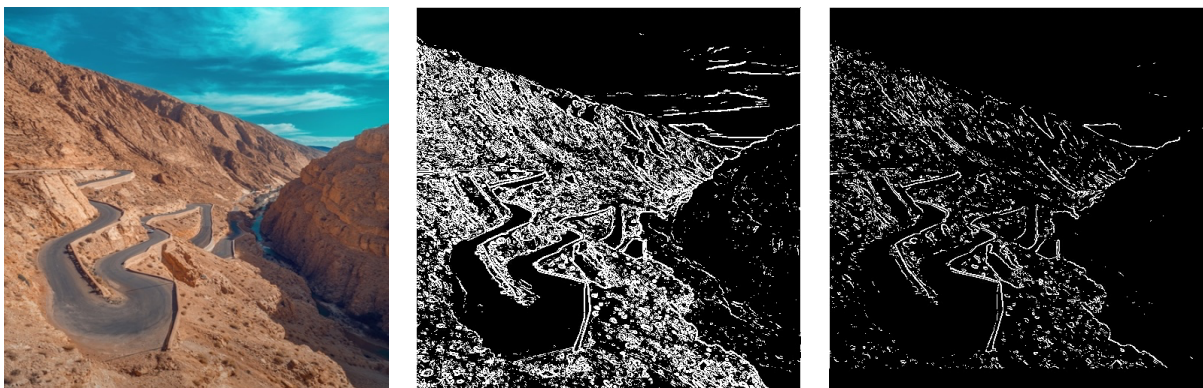
Sobelov algoritam za otkrivanje rubova implementiramo u dva koraka:

1. **Izračun gradijenta:** U stanju *processing*, modul računa gradijente u vodoravnom (*sobel_x*) i okomitom (*sobel_y*) smjeru. Oni se izračunavaju pomoću 3x3 matrica, gdje se gradijent svakog piksela računa na temelju njegovih susjednih piksela. Za rubne slučajeve kao što su prvi i zadnji stupac piksela (lijevi i desni rub slike), primjenjuju se posebni uvjeti za rukovanje graničnim pikselima, odnosno primjenjujemo “*zero padding*” metodu na rubove naše slike.
2. **Izračun magnitude:** Nakon izračuna *sobel_x* i *sobel_y*, modul računa veličinu gradijenta (*sobel_xy*) zbrajanjem apsolutnih vrijednosti *sobel_x* i *sobel_y*. Time dobijemo tzv. *threshold*. Ova veličina služi određivanju snage rubnih piksela, odnosno praga koji definira hoće li neki piksel biti bijel ili crn na obrađenoj slici. Rezultirajuće vrijednosti gradijenta zatim se zapisuju u datoteku (*output_bin.txt*) pomoću sistemske funkcije *\$fwrite*. Na slikama ispod možemo vidjeti primjere gdje smo mijenjali vrijednost *thresholda* kako bi detektirali više ili manje rubova.



Slika 6. (s lijeva na desno) - originalna slika, obrađena slika s thresholdom 127, obrađena slika s thresholdom 255, obrađena slika s thresholdom 319

Zaključujemo da je razina tresholda bitna stavka koja bi se trebala prilagođavati za svaku sliku ovisno o svjetlini slike i broju rubnih linija koje bi dobili obrađivanjem takve slike. Na primjer, slika u pustinji (*Slika 7.*) mnogo je svjetlija nego *Slika 6.*, a s istom threshold vrijednošću daje nam manje rubova, odnosno detalja na obrađenoj slici. Stoga, to može biti jedan od parametara koji utječe na broj otkrivenih rubnih linija. U stvarnim primjenama mogao bi se dodati „okidač“ koji bi prilagođavao vrijednost tresholda prema broju detektiranih rubova na slici.



Slika 7. (s lijeva na desno) - originalna slika, obrađena slika s *thresholdom* 127, obrađena slika s *thresholdom* 255

Treba istaknuti kako je kod realiziran kao automat s četiri stanja:

- **init:** Modul se pokreće u *init* stanju, resetirajući interne brojače i pripremajući sustav za obradu slike.
- **loading:** Linije slike se učitavaju iz *original_image* u line buffere (*ln0*, *ln1*, *ln2*). Gornji i donji redovi popunjeni su nulama za obradu rubnih piksela (*zero padding*), a kako se svaki redak učitava, modul se priprema za sljedeće stanje.
- **processing:** U ovom stanju, kao što samo ime kaže, Sobel operator primjenjuje se na podatke o pikselima pohranjene u line bufferima. Računa se gradijent intenziteta piksela za svaku poziciju, a rezultat se zapisuje u izlaznu datoteku. Sustav obrađuje piksele uzastopce u svakoj liniji.
- **fin:** Nakon što su svi pikseli obrađeni, modul ulazi u stanje *fin* gdje zatvara izlaznu datoteku i zaustavlja rad.

Ukratko, ova simulacija čita ulaznu sliku, primjenjuje Sobel algoritam za otkrivanje rubova i zapisuje obrađene slikovne podatke u izlaznu datoteku. Tijekom procesa, modul održava brojače (*line_counter_reg*, *byte_counter_reg*) za praćenje rednog broja trenutne linije i piksela koji se obrađuju.

Izlaznu datoteku .txt formata potrebno je obraditi pomoću Python skripte koja će binarne vrijednosti podataka pretvoriti u piksele slike .jpg formata, pogodne za jednostavno promatranje rezultata rada simulacije.

U završnoj fazi rada automatiziran je proces pokretanja potrebnih Python skripti i modula simulacije, čime je korisničko iskustvo znatno olakšano. Čitavom procesu pristupa se putem *bash*

skripte *run_sobel.sh*. Pri pokretanju, program samostalno uzima niz slika iz foldera *original_images*, jednu po jednu. Vršiti pretvorbu u tekstualnu datoteku, pokreće simulaciju te izlazne podatke pretvara u sliku .jpg formata, pohranjene u datoteci *output_images*. Dodatno, program primjenjuje više različitih *threshold* vrijednosti na istu ulaznu sliku, što dodatno olakšava promatranje utjecaja tog važnog parametra na broj detektiranih rubova. Navedene funkcionalnosti omogućene su korištenjem *bash* naredbi namijenjene za formatiranje teksta, čime je direktno moguće mijenjati određene linije koda već gotovog modula.

Iako ovaj simulacijski kod pokazuje jednostavnost primjene tehnika obrade slike na FPGA pločici, neke njegove komponente mogle bi stvarati probleme prilikom pokušaja implementacije. Pri svakom punjenju line buffer-a bajt po bajt, stvara se jedan multiplekser. To dovodi do zaključka kako će broj generiranih multipleksera biti jednak ukupnom broju piksela slike (512 x 512 multipleksera). Ova poteškoću potrebno je riješiti primjenom shift registara kojim bi se line buffere moglo „fiksirati“, a slika bi se „pomicala“ bajt po bajt. U trenutnom kodu, line bufferi su definirani kao zasebne memorije od 512 lokacija, no novi pristup zahtijevao bi pohranu svih bajtova slike u jednom velikom nizu, što također nije računalno učinkovito. Korištenjem *for* petlje, proces punjenja 3 line buffer-a bi se mogao paralelizirati, ali treba biti oprezan pri korištenju *for* petlje u implementaciji te znati kako njeno djelovanje utječe na rad i performanse sklopa. Ulaznu sliku bi se moglo učitavati liniju po liniju u simulacijski modul, čime bi broj generiranih multipleksera znatno pao te bi se približili izvornoj ideji punjenja line buffera.

4. ZAKLJUČAK

U ovom radu istraživalo se područje detekcije rubova, ključne tehnike u računalnom vidu i obradi slike. Detekcija rubova služi kao temeljni alat za identifikaciju granica objekata unutar slika, čime olakšava zadatke poput segmentacije slike, prepoznavanja uzoraka i analize scena.

Problemi na koje smo naišli:

- pronaći način kompenzacije učestalih grešaka pri slanju piksela UART-om – riješeno međusklopom
- korištenje ideje 3 line buffer-a te kreiranje nesmetane komunikacije s ulaznim FIFO međusprennikom
- logika sklopa nadmašuje resurse korištene pločice – mogući razlozi: nemogućnost implementacije line buffera veličine 512 bajtova, korištenje slike prevelikih dimenzija
- problemi u prikazu obrađene slike na VGA ekranu, nemogućnost implementacije idejnog rješenja na pločicu
- generiranje velikog broja multipleksera pri umetanju bajtova u line buffer-e (u simulacijskom modulu)

Moguća unaprjeđenja projekta:

- optimizacija koda te ispravak mogućih mana kako bi se cilj projekta ispunio do kraja
- korištenje drugih resursa pločice kako bi se postigao stabilniji i brži prijenos podataka (DDR SDRAM memorija, korištenje Ethernet sučelja pri slanju informacija itd.)
- osmisliti algoritam za adaptivnu promjenu granične vrijednosti (*engl. threshold*) u Sobel algoritmu s obzirom na sadržaj, svjetlinu slike ili broj detektiranih rubova
- slanje i obrada niza slika, obrada videozapisa u realnom vremenu (web kamera kao ulaz)

5. LITERATURA

- [1] „Image Edge Detection Operators in Digital Image Processing“, s Interneta, https://www.geeksforgeeks.org/image-edge-detection-operators-in-digital-image-processing/?ref=header_search , 20. kolovoza 2024.
- [2] Baareh, A.K.M. i dr.: „Performance Evaluation of Edge Detection Using Sobel, Homogeneity and Prewitt Algorithms“, Journal of Software Engineering and Applications, pp. 537-551. 2018.
- [3] Schiel, J., Bainbridge-Smith, A.: „Efficient Edge Detection on Low-Cost FPGAs“, arXiv preprint arXiv, 2015.
- [4] Park, C., Kim, H.: „FPGA implementation for real time SOBEL Edge Detector block using 3-Line buffers“, Journal of IKEEE, pp. 10–17, 2015.
- [5] Upadhyaya, B. K., Chakraborty, D.: „FPGA Implementation of Gradient Based Edge Detection Algorithms for Real Time Image“, 2018 2nd International Conference on Trends in Electronics and Informatics (ICOEI), 2018.
- [6] Jingcheng, S., Zhengyan, W., Zenggang, L.: „Implementation of Sobel Edge Detection algorithm and VGA display based on FPGA“, 2019 IEEE 4th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC 2019), 2019.
- [7] Martin, M.: „Detekcija linija voznih traka na slici pomoću FPGA (Diplomski rad)“, Sveučilište Josipa Jurja Strossmayera u Osijeku, Fakultet elektrotehnike, računarstva i informacijskih tehnologija Osijek, 2020., preuzeto s <https://urn.nsk.hr/urn:nbn:hr:200:127650>
- [8] Aranda, L. A., Reviriego, P., Maestro, J. A.: „Protecting Image Processing Pipelines against Configuration Memory Errors in SRAM-Based FPGAs“ Electronics, 7(11), 322., 2018.
- [9] Dhote, S. i dr.: „Using FPGA-SoC interface for low cost IoT based image processing“, 2016 Intl. Conference on Advances in Computing, Communications and Informatics (ICACCI), 2016.
- [10] Guo, L., Wu, S.: „FPGA Implementation of a Real-Time Edge Detection System Based on an Improved Canny Algorithm“, Applied Sciences, 13(2), 870., 2023.
- [11] M. Poorani, R. Kurunjimalar: "Design implementation of UART and SPI in single FGPA," 2016 10th International Conference on Intelligent Systems and Control (ISCO), Coimbatore, India, pp. 1-5, 2016.
- [12] Miao, Y., Gao, S., Liu, J., & Xu, J.: „Design of Edge Detection System Based on FPGA“, 2021 International Conference on Artificial Intelligence and Electromechanical Automation (AIEA), 2021.
- [13] Priručnik „Spartan-3 FPGA Family data sheet (DS099)“, s Interneta, <https://docs.amd.com/v/u/en-US/ds099>, 15. rujna 2024.

- [14] Priručnik „Spartan-3E FPGA Family Data Sheet (DS312)“, s Interneta, <https://docs.amd.com/v/u/en-US/ds312>, 15. rujna 2024.
- [15] Github repozitorij „PDS_SEMINAR“, s Interneta, https://github.com/70nKA/PDS_SEMINAR.git, 14. rujna 2024.
- [16] Github repozitorij „PDS_SIM“, s Interneta, https://github.com/70nKA/PDS_SIM.git, 14. rujna 2024.

DODATAK – pregled programskog koda

IDEJNO RJEŠENJE

Zbog obujma projekta te brojnih modula koje posjeduje, sav kod moguće je pronaći na Github repozitoriju [PDS_SEMINAR](#).

SIMULACIJA

Zbog obujma projekta te brojnih modula koje posjeduje, kod simulacije moguće je pronaći na Github repozitoriju [PDS_SIM](#).

BASH SKRIPTA (za automatizirano pokretanje programa)

run_sobel.sh

```
-----  
#!/bin/bash  
  
# Specify the directory containing the input .jpg files  
in_dir="original_images/"  
# Specify the directory containing the output .jpg files  
out_dir="output_images"  
  
main_script="main/sobel_main.v"  
threshold=(63 127 191 255 319)  
picture_color=("bw" "wb")  
sobel_in="img_bin.txt"  
sobel_out="sobel_img_bin.txt"  
  
# Use find to get the list of .jpg files and save them to an array  
mapfile -t jpg_files < <(find "$in_dir" -type f -name "*.jpg")  
  
# for loop which goes through all input files in original_images folder
```

```

for file in "${jpg_files[@]}"
do
    # for loop which goes through threshold options defined in threshold array
    for value in "${threshold[@]}"
    do
        # for loop which goes through color options defined in picture_color array
        for color in "${picture_color[@]}"
        do
            # command which updates sobel threshold in the sobel_main.v file
            sed -i "179s/if(sobel_xy > [0-9]\{1,4\})/if(sobel_xy > $value)/" $main_script

            if [ "$color" = "bw" ]; then
                # command which updates picture color to black and white in the sobel_main.v file
                sed -i '180s/00000000/11111111/' $main_script
                sed -i '182s/11111111/00000000/' $main_script
            else
                # command which updates picture color to white and black in the sobel_main.v file
                sed -i '180s/11111111/00000000/' $main_script
                sed -i '182s/00000000/11111111/' $main_script
            fi

            # creates filename for the output file: input_image_name + threshold + color
            basename=$(basename "$file" .jpg)
            file_out="${out_dir}/${basename}_${value}_${color}.jpg"

            # runs python script which converts jpt image to binary file
            python36 ./jpg_conversion/jpg2bin $file $sobel_in

            # compiles the overwritten code
            fuse -prj *.prj -top main_test -o main_test_sim.exe

            # runs the simulation
            ./main_test_sim.exe -tclbatch simulation.tcl

            # converts the binary sobel output to jpg

```

```
python36 ./jpg_conversion/bin2jpg $sobel_out "$file_out"
done
done
done
```

PYTHON SKRIPTA

jpg2bin.py

```
-----

import sys
from PIL import Image
import numpy as np

def convert_jpg_to_binary_txt(input_image_path, output_txt_path):
    # Otvranje ulazne slike .jpg formata
    img = Image.open(input_image_path)

    # Rezanje slike na dimenzije 512x512 piksela
    img_resized = img.resize((512, 512))

    # Pretvorba slike u crno-bijeli format
    img_gray = img_resized.convert('L')

    # Pretvorba slike u numpy niz (sadrži vrijednosti piksela)
    img_data = np.array(img_gray)

    # Zapis vrijednosti piksela u tekstualnu datoteku (8-bitni binarni format)
    with open(output_txt_path, 'w') as file:
        for row in img_data:
            for pixel in row:
                binary_pixel = format(pixel, '08b')
                file.write(binary_pixel + '\n')
```

```
print(f"Pixel data saved as 8-bit binary text: {output_txt_path}")
```

```
# Glavni dio programa
```

```
if __name__ == '__main__':
```

```
    if len(sys.argv) != 3:
```

```
        print("Usage: python script_name.py <input_image_path> <output_txt_path>")
```

```
    else:
```

```
        input_image = sys.argv[1] # prima putanju ulazne slike iz terminala
```

```
        output_txt = sys.argv[2] # prima putanju izlazne txt datoteke iz terminala
```

```
        convert_jpg_to_binary_txt(input_image, output_txt)
```

bin2jpg.py

```
-----
```

```
import sys
```

```
from PIL import Image
```

```
import numpy as np
```

```
def create_jpg_from_raw_with_padding(raw_bytes_file, output_file, width=512, height=512):
```

```
    # Čitanje binarnih podataka
```

```
    pixel_data = []
```

```
    with open(raw_bytes_file, 'r') as f:
```

```
        for line in f:
```

```
            try:
```

```
                # Pretvorba binarnog stringa u cijeli broj i dodavanje na piksel podatke
```

```
                byte = int(line.strip(), 2) # Base 2 for binary
```

```
                pixel_data.append(byte)
```

```
            except ValueError as e:
```

```
                print("Error converting line '{}': {}".format(line.strip(), e))
```

```
                continue # Preskakanje bilo koje linije koja uzrokuje grešku
```

```
# Računanje potrebnog broja piksela
```

```
required_size = width * height
```

```
# Dodavanje padding-a ako su podatci kraći od traženog
```

```
if len(pixel_data) < required_size:
```

```
    padding_size = required_size - len(pixel_data)
```

```
    print(f"Padding with {padding_size} bytes.")
```

```
    pixel_data.extend([0] * padding_size)
```

```
# Rezanje podataka ako prelaze traženu veličinu
```

```
elif len(pixel_data) > required_size:
```

```
    print("Warning: Pixel data size exceeds the required image size, trimming extra bytes.")
```

```
    pixel_data = pixel_data[:required_size]
```

```
# Pretvorba piksel podataka u numpy niz
```

```
image_array = np.array(pixel_data, dtype=np.uint8).reshape((height, width))
```

```
# Kreiranje slike crno-bijelog formata
```

```
image = Image.fromarray(image_array, mode='L')
```

```
# Pohranjivanje slike u .jpg formatu
```

```
image.save(output_file, 'JPEG')
```

```
# Glavni dio programa
```

```
if __name__ == '__main__':
```

```
    if len(sys.argv) != 3:
```

```
        print("Usage: python script_name.py <raw_bytes_file> <output_file>")
```

```
    else:
```

```
        raw_bytes_file = sys.argv[1]
```

```
output_file = sys.argv[2]  
create_jpg_from_raw_with_padding(raw_bytes_file, output_file)
```